

Cours sur le binaire

1. Introduction

L'informatique repose sur deux constats simples

- Il est relativement aisé de concevoir des systèmes ne gérant que deux états à grande vitesse
 - 0 ou 1
 - Vrai ou faux → True or False
 - Haut ou bas → High or Low
 - Ouvert ou fermé
- Ces deux états, justement, peuvent à eux seuls devenir l'image de toutes les grandeurs physiques de la planète

Exemples :

21°C peut-être représenté par $(0010\ 0001)_2$

100Km/h peut-être représenté par $(00110100)_2$

www.google.fr → $(216.58.213.131)_{10}$ → $(11011000\ 0111010\ 11010101\ 10000011)_2$

Deux remarques essentielles s'imposent :

- **Plus il y a d'informations binaires de base (bit = binary digit) plus l'image représentative de la grandeur est fine (nombre de possibilités)**
- **Plus la vitesse de cette image est rapide plus il y a d'images transmises en un temps donné.**

Les 3 bases de travail sont :

- Le décimal (base de calcul utilisée par les humains)
- Le binaire (base de calcul utilisée par la machine informatique)
- L'héxadécimal (qui est une version compressée du binaire)

Une partie du programme de l'option SIN consiste à permettre la communication entre l'homme et la machine et inversement.

On utilisera pour cela des langages informatiques qui réaliseront cette communication.

2. Bases

Les 3 bases utilisées en informatique sont :

- le décimal (base 10)
- le binaire (base 2)
- l'hexadécimal (base 16)

Base n = n éléments différents

Base 2 $\rightarrow$ 0,1

Base 10 $\rightarrow$ 0,1,2,3,4,5,6,7,8,9

Base 16 $\rightarrow$ 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

3. Convention d'écriture

110 en base 2 $\rightarrow$ $(110)_2$

110 en base 10 $\rightarrow$ $(110)_{10}$ ou 110

110 en base 16 $\rightarrow$ $(110)_{16}$

4. Relation entre bases

Passage base 10 $\rightarrow$ base 2

On divise par 2 le nombre décimal et on récupère les restes des divisions successives pour former le nombre binaire

Exemple : recherche de l'équivalent binaire de $(240)_{10}$

$$\frac{240}{2} = 120 \text{ reste } 0$$

$$\frac{120}{2} = 60 \text{ reste } 0$$

$$\frac{60}{2} = 30 \text{ reste } 0$$

$$\frac{30}{2} = 15 \text{ reste } 0$$

$$\frac{15}{2} = 7 \text{ reste } 1$$

$$\frac{7}{2} = 3 \text{ reste } 1$$

$$\frac{3}{2} = 1 \text{ reste } 1$$

$$\frac{1}{2} = 0 \text{ reste } 1$$

Sens de lecture

Résultat $(240)_{10} = (1111\ 0000)_2$

Le bit le plus à gauche est appelé MSB (Most Significatif Bit)

Le bit le plus à droite est appelé LSB (Least Significatif Bit)

4 bits côte à côte forme un **quartet**

8 bits côte à côte forme 1 **octet**, 16 bits côte à côte forme 2 octets etc...

Donc la valeur $(1111\ 0000)_2$ correspond à 1 octet ou 2 quartets

Dans les langages de programmation on verra apparaitre :

- 8 bits $\rightarrow$ 1 octets
- 16 bits $\rightarrow$ 1 mot (word) ou int
- 32 bits $\rightarrow$ mot long (long word) ou int32
- 64 bits $\rightarrow$ mot long long (long long word) ou int64

Passage base 2 $\rightarrow$ base 10

Le passage de la base 2 à la base 10 s'effectue en utilisant la forme polynomiale

$$(N)_2 \rightarrow a_n \cdot x^n + a_{n-1} \cdot x^{n-1} + a_{n-2} \cdot x^{n-2} + \dots + a_1 \cdot x^1 + a_0 \cdot x^0$$

Reprenons l'exemple précédent

$(1111\ 0000)_2$ contient 8 chiffres de rang 7(MSB) au rang 0(LSB) en base 2

Donc nous pouvons écrire ce nombre de la manière suivante :

$$a_7.x^7 + a_6.x^6 + a_5.x^5 + a_4.x^4 + a_3.x^3 + a_2.x^2 + a_1.x^1 + a_0.x^0$$

a_n représente la valeur du rang

x représente la base

donc : $(1111\ 0000)_2 = 1.2^7 + 1.2^6 + 1.2^5 + 1.2^4 + 0.2^3 + 0.2^2 + 0.2^1 + 0.2^0 = (240)_{10}$

$$\text{poids d'un octet} \rightarrow 128 + 64 + 32 + 16 + 8 + 4 + 2 + 1$$

Passage base 2 $\rightarrow$ base 16

Le principe est simple il suffit de remplacer chaque quartet par son équivalent hexadécimal

Tableau des équivalences à **apprendre par cœur**

Binaire	Hexadécimal	décimal
0000	0	0
0001	1	1
0010	2	2
0011	3	3
0100	4	4
0101	5	5
0110	6	6
0111	7	7
1000	8	8
1001	9	9
1010	A	10
1011	B	11
1100	C	12
1101	D	13
1110	E	14
1111	F	15

Exemple :

$$(1111\ 0010)_2 = (F2)_{16}$$

$$(1111\ 0010\ 1100\ 0101)_2 = (F2C5)_{16}$$

Passage base 16 → base 2

Le principe est l'inverse du précédent, on convertit chacun des chiffres du nombre hexadécimal en utilisant la table

Exemple :

$$(ABCD)_{16} = (1010\ 1011\ 1100\ 1101)_2$$

Passage base 10 → base 16

On divise par 16 le nombre décimal et on récupère les restes des divisions successives pour former le nombre hexadécimal

Exemple :

$$(240)_{10} \rightarrow$$

$$\frac{240}{16} = 15 \text{ reste } 0$$

$$\frac{15}{16} = 0 \text{ reste } 15$$

Le résultat est $(15\ 0)_{16} = (F0)_{16}$

Passage base 16 → base 10

Le passage de la base 16 à la base 10 s'effectue en utilisant la forme polynomiale

Exemple :

$$(F0)_{16} \rightarrow (N)_{10} = 15 \cdot 16^1 + 0 \cdot 16^0 = 240$$

Opération logique

Addition

Une addition s'effectue en additionnant chacun des bits entre eux en tenant compte de la retenue

Exemple :

		1	1	1	1	1	1	1	
		1	1	1	1	0	1	0	1
	+	1	0	1	0	1	1	1	1
	1	1	0	1	0	0	1	0	0

Soustraction

La soustraction nécessite de comprendre les nombres signés

Prenons l'exemple suivant :

- *Comment faire pour réaliser l'opération 15 – 7 ?*

Le principe est le suivant :

- 15 est un nombre positif (inchangé)
- 7 est précédé du signe moins et devient donc négatif
 - Il faut le changer

On utilise le complément à deux pour réaliser l'opération

Le complément à deux consiste à inverser tous les bits d'un nombre binaire et à ajouter 1

Exemple :

-7	→ s'écrit d'abord	7 = 0000 0111
	→ on inverse les bits	= 1111 1000
	→ on ajoute 1	= 1111 1001

On réalise l'opération :

15	→	0000 1111
<u>-7</u>	→ 0000 0111 → 1111 1000 →	<u>1111 1001</u>
8		0000 1000 (exact)

Multiplication

$$15 * 3 = 45$$

15 →	0000 1111
------	-----------

7 →	<u>0000 0011</u>
-----	------------------

0000 1111

00001 111

0010 1101